Identifying Spam Tests Through Machine Learning

KJ Monahan

2026-03-18

Introduction

Your phone dings. You check your texts, expecting to see a message from a friend or family member, but instead, you see an ad. Spam text messages remain a common annoyance. A 2025 survey by Orbit Media Studios found that 29% of respondents reported receiving spam messages daily or multiple times a day, with another 28% reporting receiving them weekly (2025). Spam text messages can even be harmful. A Pew Research Center survey in 2025 found that 61% of U.S. adults reported receiving spam text messages at least weekly that attempted to get the receiver's personal information (2025).

The SMS Spam Collection dataset (Almeida and María Gómez Hidalgo 2011) offers a set of spam text messages and genuine text messages ("ham") that can be used to build and test models to identify spam messages. The dataset consists of two variables: the classification ("spam" or "ham") and the text of the message. The dataset was released in 2011, and 5,574 messages, 747 of which are spam and 4,827 of which are ham (Almeida, María Gómez Hidalgo, and Yamakami n.d.).

One important consideration when comparing models is the cost of a false positive (identifying a legitimate message as spam) versus the cost of a false negative (identifying an unwanted message as ham). Text messages can be used for time-sensitive communication, with the expectation that the receiver will receive an alert immediately. Real-world examples range from letting someone know you've arrived to two-factor authentication codes that expire after a short period of time. CTIA, an organization representing wireless companies in America, also emphasizes the urgency of some communications, such as flight changes or fraud alerts, in their best practice guidelines for members (2023). Governmental organizations can use text messaging to send emergency notifications, such as an active tornado warning or a water main breakage requiring a temporary boil order. Missing an urgent, or less time-sensitive but still meaningful message, can cause harm when people rely on text messaging for communication. CTIA's 2025 annual survey found that Americans exchanged nearly 2.2 trillion text messages in 2025 (2025a).

However, failing to identify a spam message could be harmful if the message is one of the many that attempts to defraud the receiver or trick them into handing over personal information. The Orbit Media Studios survey found that while ads were still the most common form of spam text messages, 37% of respondents said that the spam text messages they received asked for personal information (2025). While the risk is real, Pew Research Center found that 71% of U.S. adults feel they know at least a fair amount about how to avoid falling for an online scam (a category that includes spam text messages) (2025). The FTC, among other organizations, provides guidance on how to recognize and report text message scams (2022), which may help decrease the likelihood that someone seeing a spam message is actively harmed.

Considering the trade-offs, I determined that recall should be weighted more heavily than precision–it was more important that someone not miss a real text message than receive a spam message. To compare models, I calculated the F value, which combines precision and recall, with a beta of 0.75, weighting precision 0.75 times that of recall (Irizarry, n.d.a; "What Is F-Beta Score?" 2025).

Final summary here

Methods and Analysis

Data Cleaning

The SMSSpamCollection file is a tab-separated text file, with one line per message. I parsed it into rows and columns using `read_tsv`.

```
lines <- readLines("data/SMSSpamCollection")
rows <- str_split(lines, "\t")
messages <- data.frame(parse_factor(map_chr(rows, 1)), parse_character(map_chr(rows, 2)))
colnames(messages) <- c("type", "message")
head(messages, 10) |> tibble()
```

```
## # A tibble: 10 x 2
##    type  message
##    <fct> <chr>
##  1 ham   Go until jurong point, crazy.. Available only in bugis n great world l~
##  2 ham   Ok lar... Joking wif u oni...
##  3 spam  Free entry in 2 a wkly comp to win FA Cup final tkts 21st May 2005. Te~
##  4 ham   U dun say so early hor... U c already then say...
##  5 ham   Nah I don't think he goes to usf, he lives around here though
##  6 spam  FreeMsg Hey there darling it's been 3 week's now and no word back! I'd~
##  7 ham   Even my brother is not like to speak with me. They treat me like aids ~
##  8 ham   As per your request 'Melle Melle (Oru Minnaminunginte Nurungu Vettam)'~
##  9 spam  WINNER!! As a valued network customer you have been selected to receiv~
## 10 spam  Had your mobile 11 months or more? U R entitled to Update to the lates~
```

When I tested splitting the messages by words to see what the most popular words used were, I saw "gt" and "lt" in the list. While "2", "ur", and "4" are all commonly-known texting abbreviations, especially in 2011, when the dataset was collected, "gt" and "lt" seemed like odd words to be sent so frequently.

```
messages |>
  unnest_tokens(word, message, to_lower=TRUE, strip_punct=TRUE) |>
  filter(!word %in% stop_words$word) |>
  count(word) |>
  top_n(10, n) |>
  mutate(word = reorder(word, n)) |>
  arrange(desc(n))
```

```
##    word   n
## 1  call 588
## 2     2 513
## 3    ur 391
## 4    gt 318
## 5     4 316
## 6    lt 316
## 7  free 283
## 8   day 221
## 9  time 218
## 10 love 207
```

Reviewing the messages that contain 'lt' or 'gt' indicates that these are HTML-encoded entities.

```
messages |>
  filter(sapply(message, function(m) {
    words <- list_c(tokenize_words(m, lowercase = TRUE, strip_punct=TRUE))
    any(c('lt', 'gt') %in% words)
  })) |>
```

```
  pull(message) |>
  head()
```

```
## [1] "Great! I hope you like your man well endowed. I am  &lt;#&gt;  inches..."
## [2] "A gram usually runs like  &lt;#&gt; , a half eighth is smarter though and gets you almost a whol
## [3] "Do you know what Mallika Sherawat did yesterday? Find out now @  &lt;URL&gt;"
## [4] "Does not operate after  &lt;#&gt;  or what"
## [5] "Turns out my friends are staying for the whole show and won't be back til ~ &lt;#&gt; , so feel
## [6] "Found it, ENC  &lt;#&gt; , where you at?"
```

Using the `textutils` package, I decoded any encoded HTML entities and stripped any HTML tags that might still have been included.

```
messages <- messages |> mutate(message=HTMLrm(HTMLdecode(message)))
messages |>
  unnest_tokens(word, message, to_lower=TRUE, strip_punct=TRUE) |>
  filter(!word %in% stop_words$word) |>
  count(word) |>
  top_n(10, n) |>
  mutate(word = reorder(word, n)) |>
  arrange(desc(n))
```

```
##    word   n
## 1  call 588
## 2     2 513
## 3    ur 391
## 4     4 316
## 5  free 283
## 6   day 221
## 7  time 211
## 8  love 207
## 9  send 196
## 10 text 193
```

Now the most popular words all make sense as words that might be used in texts.

Next I standardized phone numbers. Even in the small sample of texts reviewed, I could see at least one phone number, and I knew that spam messages might instruct someone to call or a text a certain number. The presence of "call" and "text" as commonly used words supported that. But since phone numbers can take a variety of forms, I standardized several known forms to be all digits. This would also help me later when testing whether the presence of long strings of numbers, such as phone numbers or account numbers, would be indictative of a spam message.

```
convert_phone_numbers <- function(s) {
  s |>
    # Handle formats 000-000-0000, 000.000.000, and 000 000 0000
    str_replace_all("(\\d{3})[-,\\.\\s](\\d{3})[-,\\.\\s](\\d{4})",
                    "\\1\\2\\3") |>
    # Handle format (000) 000-0000
    str_replace_all("\\((\\d{3})\\)\\s?(\\d{3})-(\\d{4})", "\\1\\2\\3")
}
messages <- messages |>
  mutate(message = convert_phone_numbers(message))
```

Analysis

Before doing any analysis, I split the data into training and test sets, so that I could look for differences between ham and spam text messages in the training set only. Knowing that, of course, the data set represents a small fraction of the spam and ham text messages sent, I wanted to use the test set as a stand in for the unknown or yet-to-be-set text messages that I couldn't analyze.

Given the overall small size of the data set, I wanted to use as much data as possible for analysis and training. However, the test set still needs to be large enough to contain meaningful data. A split of 10%-20% of the data for the test set is common (Irizarry, n.d.b). I chose 20%, so there would be a little over one thousand messages in the test set. As 2024 study proposed a minimum data set sizes of N=500-1000 to reduce overfitting (Zantvoort et al., n.d.), the remaining training set size of around 4400 messages should be sufficient.

Most Commonly Used Words I compared the mostly commonly-used words in the training set for ham messages versus spam messages.

```
##    word   n
## 1     2 266
## 2    ur 202
## 3  call 183
## 4   day 167
## 5  love 166
## 6  time 161
## 7     4 154
## 8  home 130
## 9     ü 129
## 10  lor 128
```

```
##      word   n
## 1    call 287
## 2    free 180
## 3       2 141
## 4     txt 119
## 5  mobile 110
## 6    text 109
## 7      ur 109
## 8    stop  98
## 9       4  97
## 10  claim  94
```

There are some apparent differences. "Free" is the second-most-often used words in spam texts but does not appear in the top 10 at all for ham texts. However, there is also overlap. "Call", "ur," "2", and "4" appear in both lists. Therefore, merely identifying words that are used often in spam messages are likely to result in a lot of false positives. Instead, I calculated the odds ratio for each word to identify those that were most likely to be in spam messages. (Irizarry, n.d.a)

```
## # A tibble: 15 x 4
##    word         ham  spam odds_ratio
##    <chr>      <dbl> <dbl>      <dbl>
##  1 claim          0    94       442.
##  2 prize          0    72       338.
##  3 won            0    56       263.
##  4 150p           0    55       258.
##  5 tone           0    45       211.
##  6 guaranteed     0    40       188.
##  7 18             0    37       174.
```

```
##  8 cs             0    36       169.
##  9 1000           0    33       155.
## 10 1.50           0    32       151.
## 11 500            0    32       151.
## 12 awarded        0    29       137.
## 13 150ppm         0    27       128.
## 14 ringtone       0    27       128.
## 15 5000           0    23       109.
```

This demonstrates that are definitely words that appear more often in spam messages than in ham messages. There are 7,513 unique words used in the training set. 2,087 of those words have an odds ratio greater than 1, indicating a positive correlation between the presence of that word in the message and the message being spam. 759 of the words have an odds ratio greater than 10, indicating an even stronger positive correlation. (Bobbitt 2021)

I then counted the number of times a word in the list of top spam words (those that are at least 10 times more likely to appear in a spam message than a ham message in the training set) and added that data to both the training and test sets.

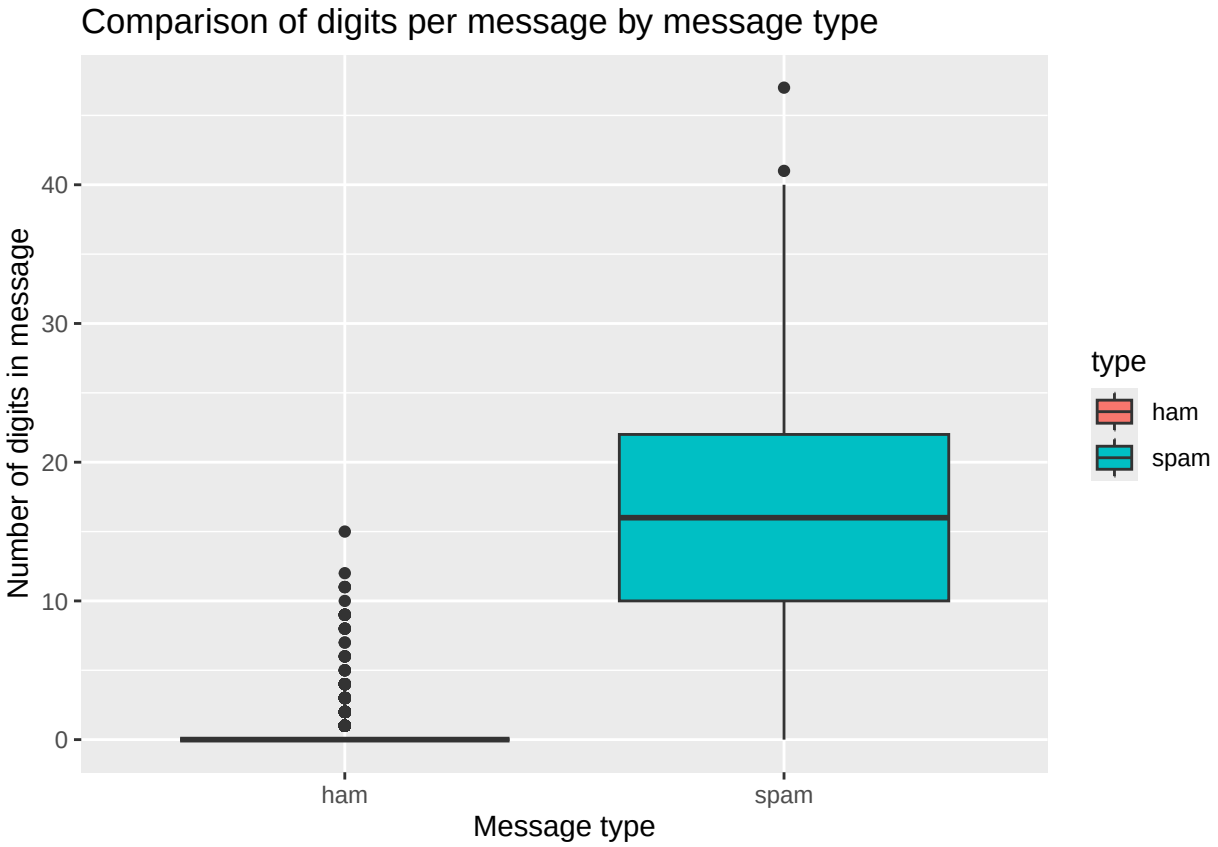
```r
# Get the words with an odds ratio of 10 or more. That's the list of words
# that are at least 10x more likely to be in a spam message than a ham message.
top_spam_words <- ham_spam_odds |> filter(odds_ratio >= 10) |> pull('word')

# Count how many times any of those words appear in the message
count_spam_words <- function(data, word_list = top_spam_words) {
  data |>
    mutate(top_spam_words_count = sapply(message, function(m) {
      words <- list_c(tokenize_words(m, lowercase = TRUE, strip_punct=TRUE))
      sum(words %in% top_spam_words)
    }))
}

train_set <- count_spam_words(train_set)
test_set <- count_spam_words(test_set)
```

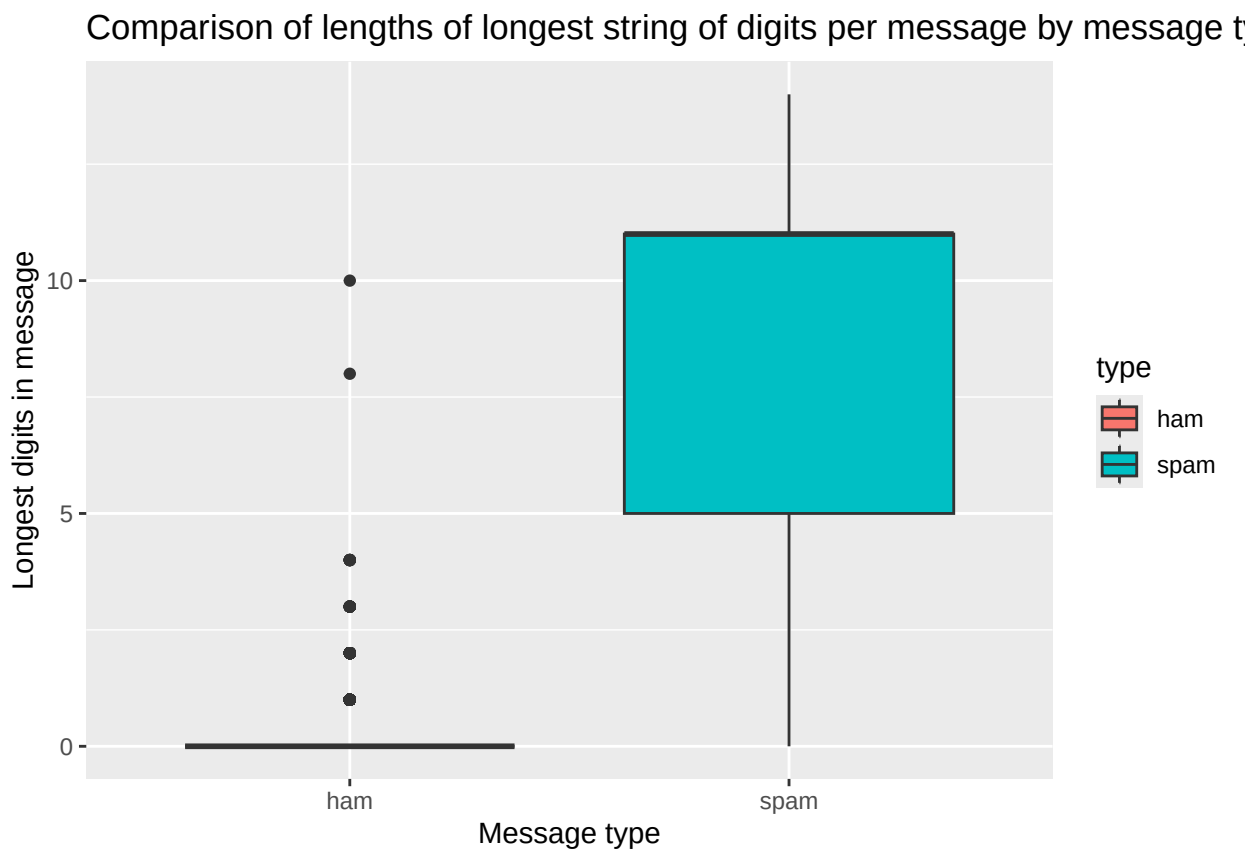
Numbers Previously I'd hypothesized that spam messages might be more likely to contain phone numbers or other long number strings, such as a (faux) bank account or package tracking number. In addition, the list of words more likely to be in spam messages contained several numbers.

First, I added two counts to my data sets: the total number of digits appearing in the message and the length of the longest unbroken string of numbers.

```r
add_number_columns <- function(data) {
  data |>
    mutate(message = convert_phone_numbers(message)) |>
    mutate(num_count = str_count(message, "\\d")) |>
    mutate(longest_number = unlist(lapply(str_match_all(message, "\\d+"), function(m) {
      if (length(m) == 0) {
        0
      } else {
        max(nchar(m[,1]))
      }
    })))
}
train_set <- add_number_columns(train_set)
test_set <- add_number_columns(test_set)
```

Spam messages tend to have both more digits overall and a longer string of digits than ham messages do.





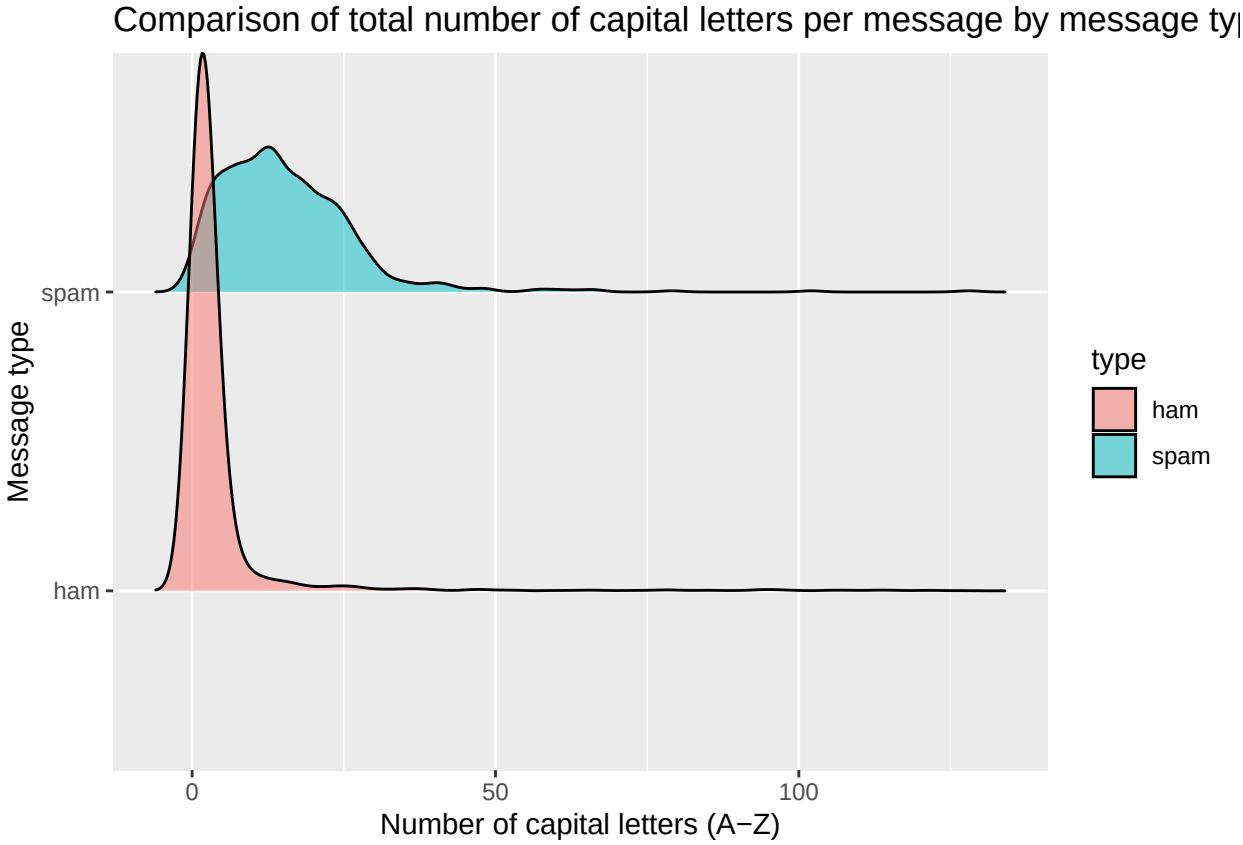
Capital Letters Like numbers, it's possible that capital letters could differ between spam and ham messages. However, with few options to convey tone over text messages (the dataset was published in 2011, which predates the global rise in emoji popularity from 2014 onward (Broni 2025)), capital letters or entire words or messages in all caps might be used by both spam messages and ham messages to convey excitement or urgency.

To see if there was any difference, I first counted both the total number of capital letters and the longest string that did not contain lowercase letters or digits. I did allow for white space and punctuation in order to detect the difference between a single word in all-caps and an entire message in all-caps, so long as the string started and ended with a capital letter (so removing trailing white space and punctuation).

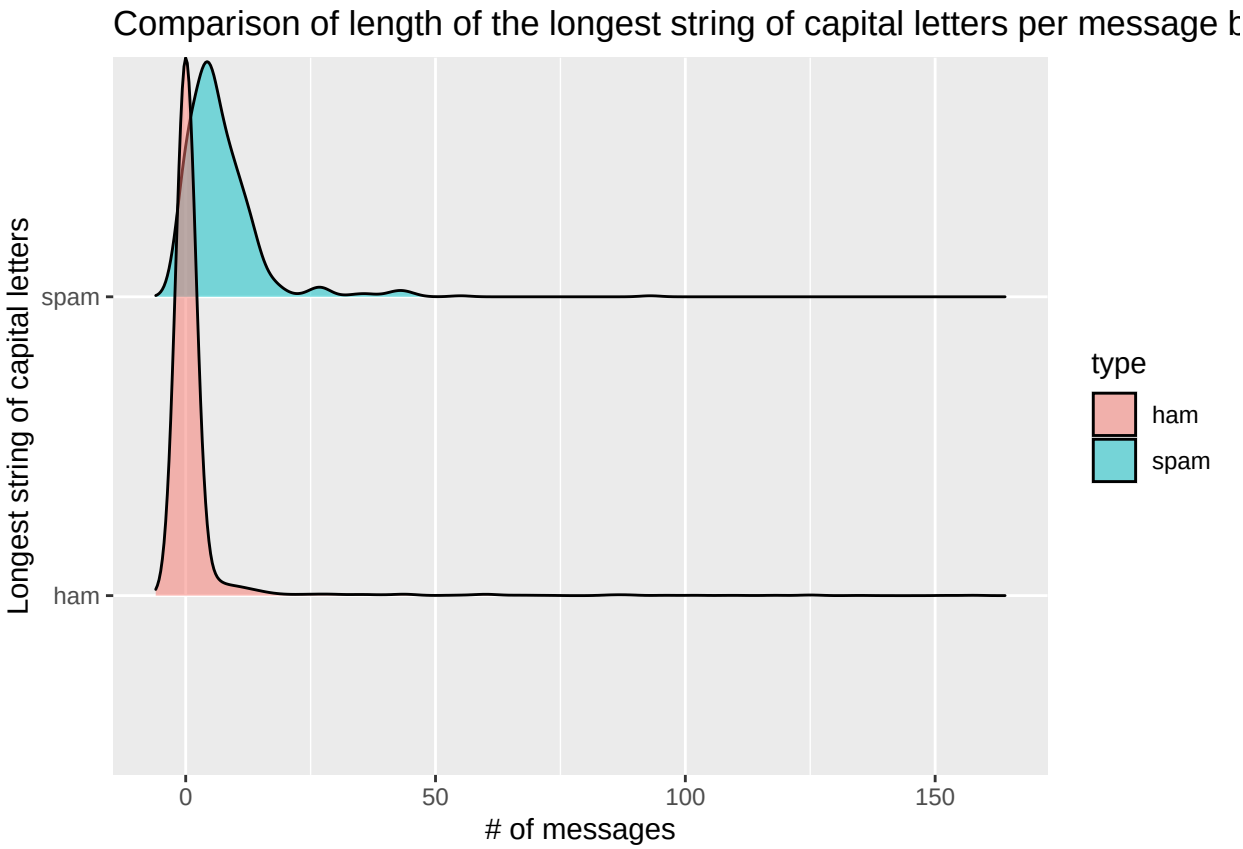
```
add_capital_columns <- function(data) {
  data |>
    mutate(capital_count = str_count(message, "[A-Z]")) |>
    mutate(longest_capitals = unlist(lapply(str_match_all(message, "[A-Z][^a-z\\d]*[A-Z]"), function(m)
      if (length(m) == 0) {
        0
      } else {
        max(nchar(m[,1]))
      }
    })))
}

train_set <- add_capital_columns(train_set)
test_set <- add_capital_columns(test_set)
```

Using a ridge plot, I can see that while most ham messages have only a few capital letters, there's a long tail.

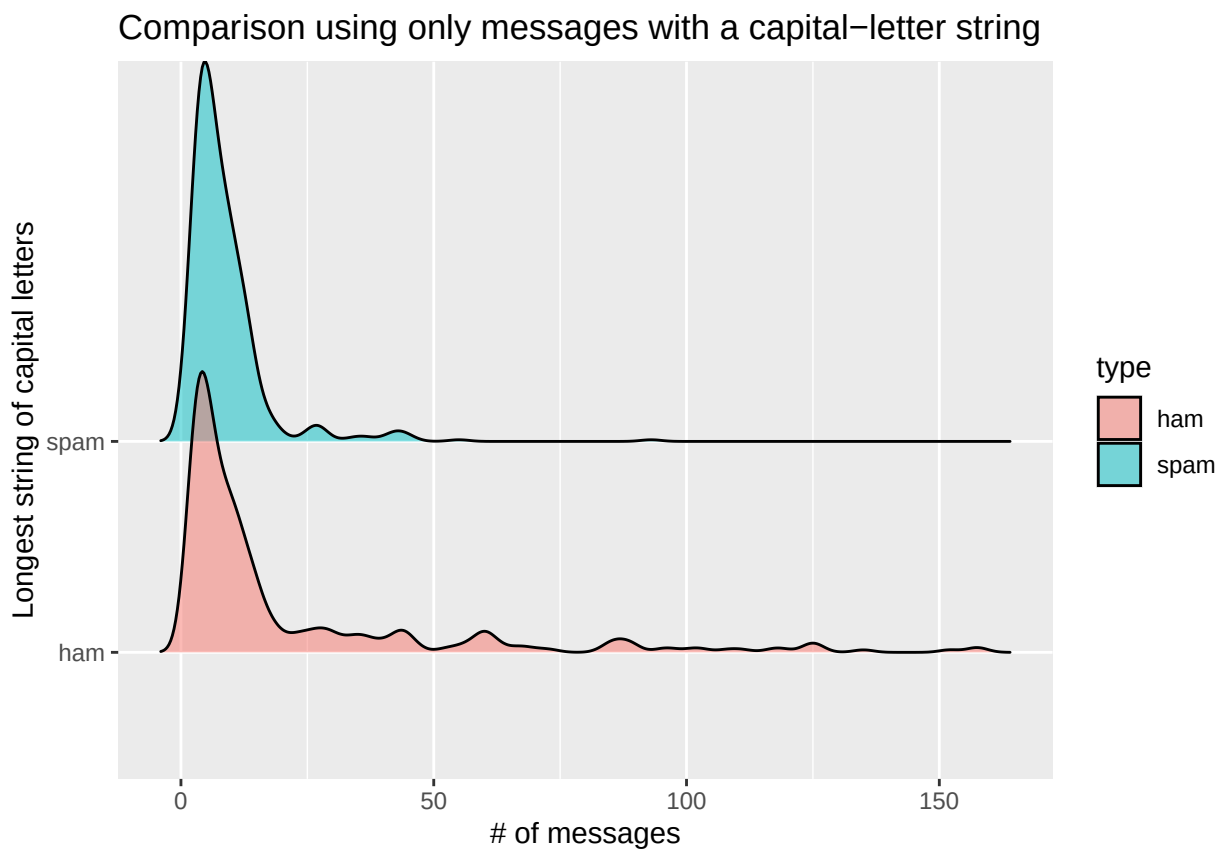
There's also overlap between spam and ham messages at the lower end of the range. Messages with 5 to 30 capitals are more likely to be spam messages, but then spam messages also have a long tail.



Comparing the length of the longest string of capital letters, there's a similar pattern. Both ham and spam messages have long tails and both overlap at the bottom of the range, with no string of two or more capital letters. However, while ham messages are much more likely to have no string than to have any length of capital letters in a row, spam messages have a wider distribution with a peak around a string of 5-10 characters.



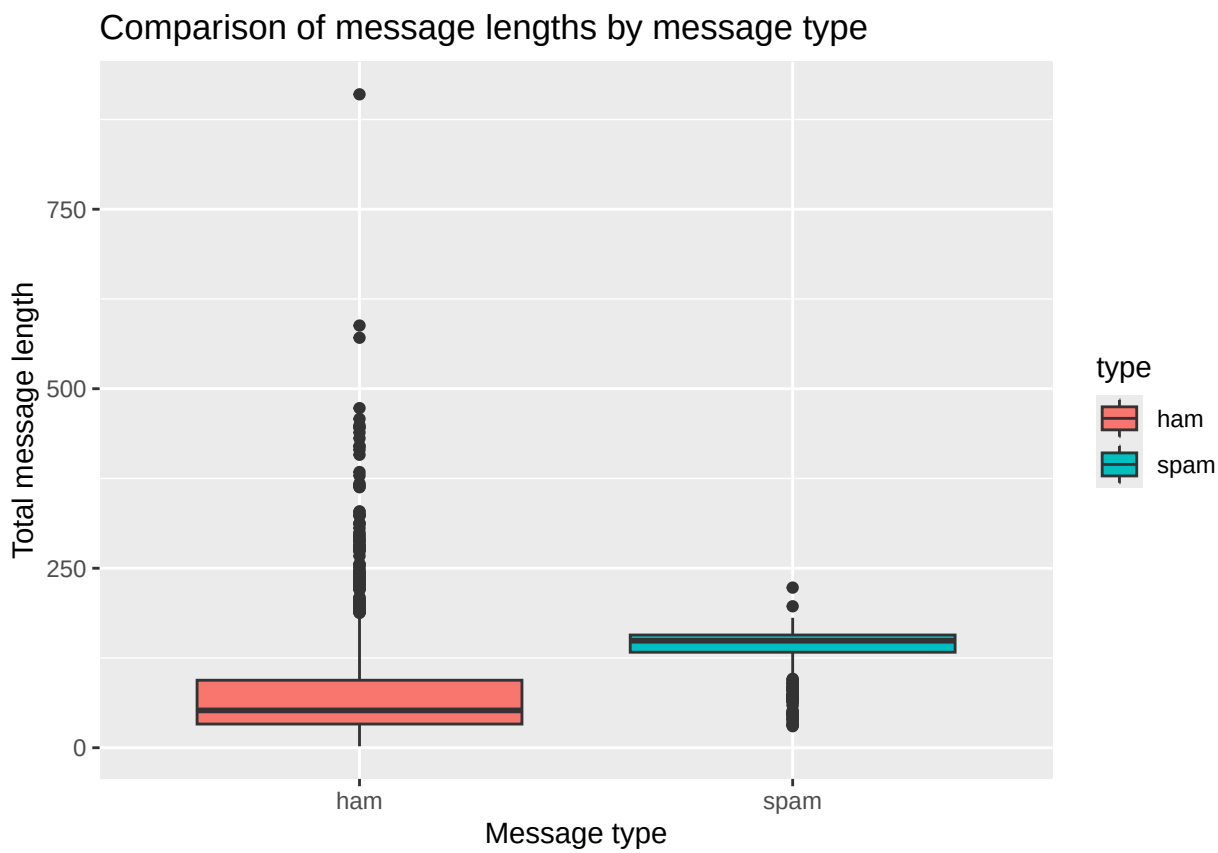
Looking only at messages where `longest_capitals` is greater than 0, I can see that most of the difference between spam and ham messages is driven by most ham messages not having a multi-capital-letter string. However, with those removed, it's more apparent that while spam messages peak at a slightly greater number, ham messages have a more substantial tail. This suggests that while having a `longest_capitals` length of 0 might give credence to a message being ham, the length of the string beyond that doesn't seem to be a clear indicator one way or another.



Message length Finally, I counted the total character length of the messages to examine if there were any differences.

```
train_set <- train_set |> mutate(msg_length = nchar(message))
test_set <- test_set |> mutate(msg_length = nchar(message))
```

I can see that, on average, ham messages are shorter than spam messages. However, once again, both types contain a number of outliers, with messages longer than 250 characters being entirely ham messages.



Model Development

To compare model performance, I wanted to consider both precision and recall, but, as discussed earlier, giving more weight to recall. I tested each model against my previously held-out test set and compared the F-value, which computes the harmonic average of precision and recall. I used a weighted harmonic average with a beta value of 0.75, so that precision was weighted 0.75 times that of recall.

Model 1: Guessing To get a baseline, I first tested an extremely simple model, guessing whether a message is spam or ham, assuming the same proportions of spam messages that are in the training set.

```
p_spam <- sum(train_set$type == 'spam') / nrow(train_set)
guess_y_hat <- sample(c('spam', 'ham'), nrow(test_set), replace = TRUE, prob = c(p_spam, 1 - p_spam)) |>
  factor(levels = levels(test_set$type))
guess_f_meas <- F_meas(
  guess_y_hat,
  reference = test_set$type,
  relevant = "spam",
  beta = 0.75
)
guess_f_meas
```

```
## [1] 0.1625065
```

Model 2: Using longest number During the analysis, I found that messages with long strings of digits are more likely to be spam. For the second model, all messages with the longest string of digits above a certain cutoff are classified as spam, all others as ham. To find the cutoff, I compared the F-value against the

training set for cutoffs ranging from 0 to 15. A cutoff of 4 provided the optimal value, so I then tested on the test set, using that cutoff.

```
# Determined through tuning on the training set
cutoff <- 4
long_number_y_hat <- ifelse(test_set$longest_number >= cutoff, "spam", "ham") |>
  factor(levels = levels(test_set$type))
long_number_f_meas <- F_meas(
  long_number_y_hat,
  reference = test_set$type,
  relevant = "spam",
  beta = 0.75
)
long_number_f_meas
```

```
## [1] 0.9226714
```

This improves performance dramatically over guessing.

Model 3: Generalized linear model Because messages are either spam or ham, which can be represented as 0 and 1, a generalized linear model or logistic regression model makes more sense than a linear regression model, which would allow for values below 0 or above 1. A logistic regression model converts the probability to log odds, calculating how much more likely it is that a message will be spam than that it will not be spam. Rather than a simple binary cutoff, I can use a linear regression model to calculate whether a message is likely spam or ham given a value.

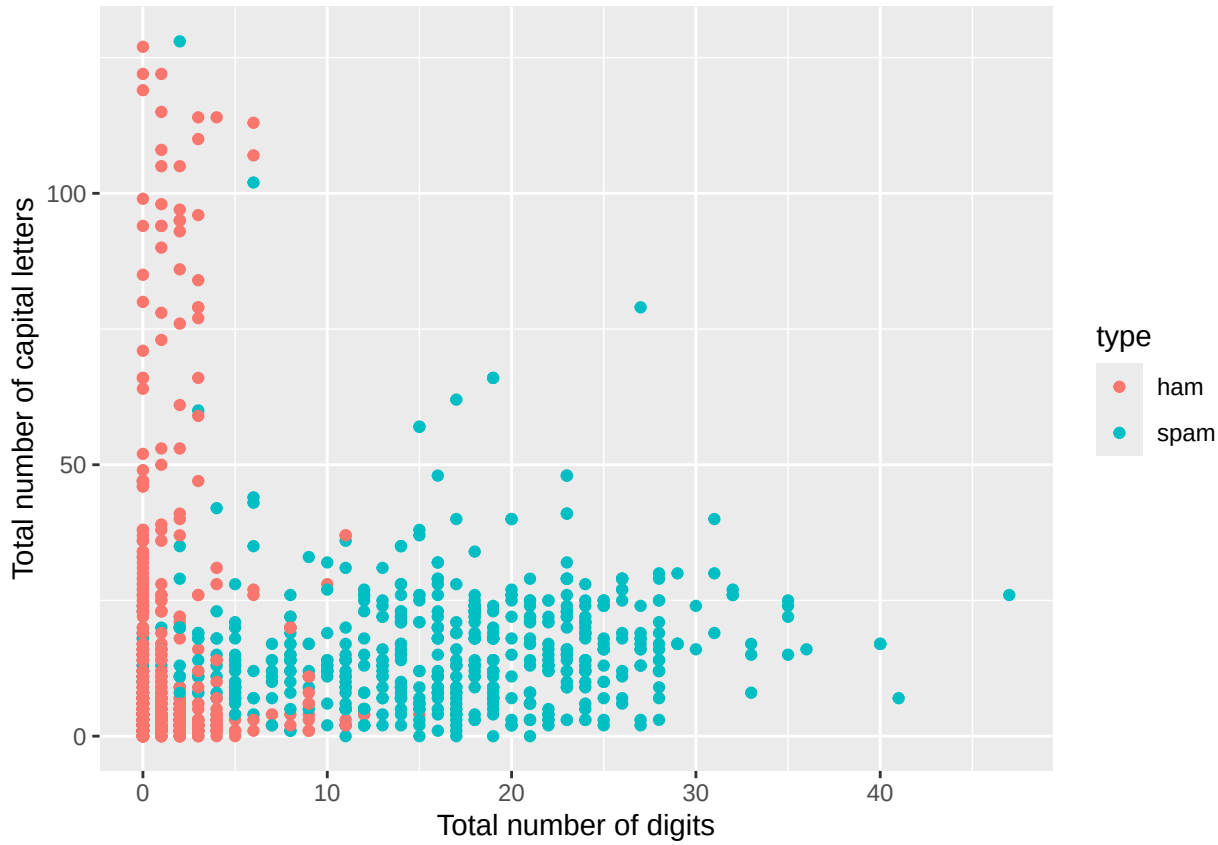
```
glm_digit_length_fit <- train_set |>
  mutate(y = as.numeric(type == "spam")) |>
  glm(y ~ longest_number, data = _, family="binomial")
# Predict the results on the test set, using the model created on the training set
# Returns the likelihood that a message is spam
glm_digit_length_p_hat <- predict(glm_digit_length_fit, newdata=test_set, type="response")
# Translate those probabilities to predictions by saying that any message with
# a greater than 50% chance of being spam is spam
glm_digit_length_y_hat <- ifelse(glm_digit_length_p_hat > 0.5, "spam", "ham") |>
  factor(levels = levels(test_set$type))
glm_digit_length_f_meas <- F_meas(
  glm_digit_length_y_hat,
  reference = test_set$type,
  relevant = "spam",
  beta = 0.75
)
glm_digit_length_f_meas
```

```
## [1] 0.8972943
```

This did less well than finding a cut-off. While messages with longer numbers are more likely to be spam, the data is approximately bivariate normal.

I also considered looking at more than one variable. For example, messages with more numbers and with more capital letters both tended to be spam messages.

```
train_set |>
  ggplot(aes(num_count, capital_count, color = type)) +
  xlab("Total number of digits") +
  ylab("Total number of capital letters") +
  geom_point()
```



While the data is mostly grouped between ham and spam messages, the relationship is not linear.

Model 4: K-Nearest Neighbor K-Nearest Neighbor (KNN) identifies the k closest data points to an item and makes the prediction based on the average of the closest data points. One advantage it has over logistic or linear regression here is that it does not assume the data is distributed in any particular way. Since ham and spam messages tend to be clustered near similar messages, predicting the classification based on the nearest other messages might yield better results. ("K-Nearest Neighbor(KNN) Algorithm" 2026)

I used the `caret` package to train the model and perform cross-validation with bootstrap samples to determine which value of k performed best. Bootstrap sampling takes a sample with replacement of the training dataset to approximate the full dataset. The `train` method does this 25 times (the default value) for each parameter and calculates the accuracy. This approximates the distribution across the full dataset. I used the same weighted F value for accuracy that has been used on models previously. Whatever parameter value gives the highest accuracy when the 25 tests are averaged together is what's used in the final model.

```
knn_fit <- train(
  type ~ .,
  method = "knn",
  data = train_set_features,
  trControl = control,
  metric = "F1",
  tuneGrid = data.frame(k = c(3, 5, 7, 9, 11))
)
knn_fit$bestTune
```

```
##   k
## 3 7
```

I then tested the final model against the test set.

```
knn_y_hat <- predict(knn_fit, newdata = test_set, type="raw")
knn_f_meas <- F_meas(
  knn_y_hat,
  reference = test_set$type,
  relevant = "spam",
  beta = 0.75
)
knn_f_meas
```

```
## [1] 0.9226714
```

Model 5: Classification Tree Another way to classify messages is to look at various combinations of factors. For instance, I've already seen that messages with long strings of numbers are likely to be spam. Messages with more than 5 capital letters in a row are likely spam unless they have more than 50 capital letters, in which case, they're likely ham.

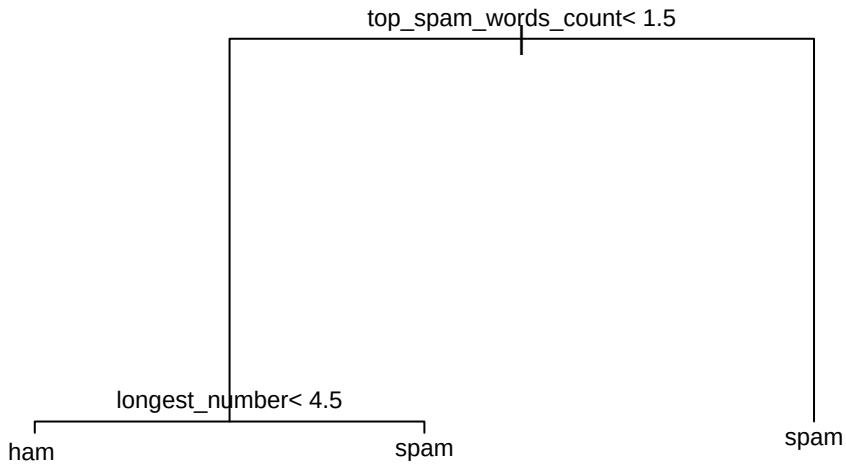
A classification tree establishes a set of rules or series of if/else conditions to determine the prediction. The complexity parameter (cp) is the value that can be tuned here to determine the optimal minimum by which adding another split must improve the model (Irizarry, n.d.b).

```
class_tree_fit <- train(
  type ~ .,
  method = "rpart",
  data=train_set_features,
  trControl = control,
  metric = "F1",
  tuneGrid = data.frame(cp = seq(0.0, 0.1, len = 25)),
)
class_tree_y_hat <- predict(class_tree_fit, newdata = test_set, type="raw")
class_tree_f_meas <- F_meas(
  class_tree_y_hat,
  reference = test_set$type,
  relevant = "spam",
  beta = 0.75
)
class_tree_f_meas
```

```
## [1] 0.9563212
```

Plotting the final model shows what variables and cut-off values were used.

```
plot(class_tree_fit$finalModel, margin = 0.1)
text(class_tree_fit$finalModel, cex = 0.75)
```



The model only needed the number of occurrences of top spam words and the length of the longest number to predict the outcome.

Model 6: Random Forest Random forest builds on the classification tree idea by randomly selecting bootstrap samples and building classification trees from the samples, using randomly selected features. The resulting trees create the "forest", and the average prediction of all the trees determines the final prediction (Irizarry, n.d.b).

```
random_forest_fit <- train(
  type ~ .,
  method = "rf",
  data = train_set_features,
  trControl = control,
  metric = "F1",
  # Tune the number of variables randomly assigned per split
  tuneGrid=data.frame(mtry=seq(1, 6))
)
random_forest_y_hat <- predict(random_forest_fit, newdata = test_set, type="raw")
random_forest_f_meas <- F_meas(
  random_forest_y_hat,
  reference = test_set$type,
  relevant = "spam",
  beta = 0.75
)
random_forest_f_meas
```

```
## [1] 0.968939
```

When reviewing the relative importance of the various factors, I can see that `top_spam_words_count` had the greatest impact. That is the same factor that was used in the first split in the classification tree model.

```
##                       MeanDecreaseGini
## top_spam_words_count        279.86764
## num_count                   206.84614
## longest_number              228.28684
## capital_count                91.57365
## longest_capitals            100.09202
## msg_length                   65.66871
```

Ensembles

After testing models one at a time, I considered whether combining predictions from multiple models could yield better results.

One challenge is that I wanted to only use models that were among the most accurate, but I could not use the previously computed accuracy results, because they were based on test data. Using the results from testing on the test set to determine which model to use would pollute my data.

Instead, I used the same bootstrap approach used for tuning the model parameters. I created five bootstrap samples from the training data, predicted the outcome for each using the existing models, and averaged the resulting F-values.

```
## # A tibble: 6 x 2
##   Models               `F-Values`
##   <chr>                     <dbl>
## 1 Guessing                  0.137
## 2 Longest number            0.920
## 3 Logistic regression       0.915
## 4 K-Nearest Neighbor        0.951
## 5 Classification Tree       0.970
## 6 Random Forest             0.983
```

Five of the models had F-values of greater than 0.9 against the training data bootstrap samples.

Voting The first ensemble model I tried was a voting method. Since Longest number and Logistic regression were both models based on the `longest_number` feature, I only used one of them. I chose Longest number because it performed better in my bootstrap sample.

This left me with an even number of models, so I had to determine how to handle ties. Since I had decided that not misidentifying ham was more important than not misidentifying spam, in case of a tie, the message was classified as ham. That meant that 3 out of the 4 models had to consider a message spam in order to classify it as such.

```r
ensemble_vote_fit <- cbind(long_number_y_hat, class_tree_y_hat, random_forest_y_hat, knn_y_hat)
ensemble_vote_y_hat <- apply(ensemble_vote_fit, 1, function(r) {
  ifelse(sum(r == 2) >= 3, 'spam', 'ham')
}) |> factor(levels = levels(test_set$type))
ensemble_vote_f_meas <- F_meas(
  ensemble_vote_y_hat,
  reference = test_set$type,
  relevant = "spam",
  beta = 0.75
)
ensemble_vote_f_meas
```

```
## [1] 0.9431225
```

Mean Probability The second ensemble model I tried was by calculating the probability of a message being spam, using the top three models: KNN, Classification Tree, and Random Forest. I took the mean of the three probabilities and used the mean probability to predict whether the message was spam or ham.

```
# Re-generate predictions to get the probability of the item being spam
knn_p_hat <- predict(knn_fit, newdata = test_set , type = "prob")['spam']
class_tree_p_hat <- predict(class_tree_fit, newdata = test_set, type = "prob")['spam']
random_forest_p_hat <- predict(random_forest_fit, newdata = test_set, type = "prob")['spam']
ensemble_mean_fit <- cbind(knn_p_hat, class_tree_p_hat, random_forest_p_hat)
# Get the mean probability across all three models. If it's greater than 0.5
# likelihood of being spam, classify it as such
ensemble_mean_y_hat <- apply(ensemble_mean_fit, 1, function(r) {
  ifelse(mean(r) > 0.5, 'spam', 'ham')
}) |> factor(levels = levels(test_set$type))
ensemble_mean_f_meas <- F_meas(
  ensemble_mean_y_hat,
  reference = test_set$type,
  relevant = "spam",
  beta = 0.75
)
ensemble_mean_f_meas
```

```
## [1] 0.966317
```

Results

While the ensemble mean approach improved the results over the classification tree and KNN models on their own, the random forest model gave the most accurate results overall.

```
## # A tibble: 8 x 2
##   Models              `F-Values`
##   <chr>                    <dbl>
## 1 Guessing                 0.163
## 2 Longest number           0.923
## 3 Logistic regression      0.897
## 4 K-Nearest Neighbor       0.923
## 5 Classification Tree      0.956
## 6 Random Forest            0.969
## 7 Ensemble Voting          0.943
## 8 Ensemble Mean            0.966
```

The full confusion matrix details that the random forest model has specificity/recall over 99% and a slightly lower sensitivity/precision. It correctly classified 99% of the messages in the test set overall and 94% of the spam messages. Importantly, only 2 of the 966 ham messages were incorrectly classified as spam.

```
confusionMatrix(random_forest_y_hat, test_set$type, positive = "spam")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction ham spam
##       ham  964    9
##       spam   2  141
##
##                Accuracy : 0.9901
##                  95% CI : (0.9824, 0.9951)
##     No Information Rate : 0.8656
```

```
##     P-Value [Acc > NIR] : < 2e-16
##
##                   Kappa : 0.9568
##
##  Mcnemar's Test P-Value : 0.07044
##
##             Sensitivity : 0.9400
##             Specificity : 0.9979
##          Pos Pred Value : 0.9860
##          Neg Pred Value : 0.9908
##              Prevalence : 0.1344
##          Detection Rate : 0.1263
##    Detection Prevalence : 0.1281
##       Balanced Accuracy : 0.9690
##
##        'Positive' Class : spam
##
```

Conclusion

Overall, a random forest model works well for classifying spam messages using factors such as the presence of words known to be used more often in spam messages, long strings of digits such as phone numbers, and numbers of digits or capital letters.

Further research could be done using a more recently-collected data set. The dataset used here was donated in 2011, and changes in technology and communication styles may have changed what makes a message identifiable as spam. Other factors that could be considered in a future study would be whether emoji usage or attachments such as photos are indicative of a message being spam or ham. In addition, the work here attempts to identify messages as spam or ham based only on the message content. Other context about the message itself, such as whether the receiver has communicated with the sender before or whether the message was sent via an automated program or another phone, could potentially further improve classification.

Almeida, Tiago, and José María Gómez Hidalgo. 2011. "SMS Spam Collection." UCI Machine Learning Repository. https://doi.org/https://doi.org/10.24432/C5CC84.

Almeida, Tiago, José María Gómez Hidalgo, and Akebo Yamakami. n.d. "Contributions to the Study of SMS Spam Filtering:New Collection and Results." In *DocEng '11: Proceedings of the 11th ACM Symposium on Document Engineering*, 259–62. Association for Computing Machinery. Accessed March 15, 2026. https://dl.acm.org/doi/epdf/10.1145/2034691.2034742.

Bobbitt, Zach. 2021. "How to Interpret Odds Ratios." Statology. August 26, 2021. https://www.statology.org/interpret-odds-ratios/.

Broni, Keith. 2025. "Major Moments In Emoji History: 1995* to 2025." *Emojipedia* (blog). July 16, 2025. https://blog.emojipedia.org/major-moments-in-emoji-history-1995-to-2025/.

Crestodina, Andy. 2025. "2025 Spam Report: The Abuse of Trust in Email and Social (Now with AI)." *Orbit Media Studios* (blog). December 2, 2025. https://www.orbitmedia.com/blog/spam-statistics/.

CTIA. 2023. "Messaging Principles and Best Practices." https://api.ctia.org/wp-content/uploads/2023/05/230523-CTIA-Messaging-Principles-and-Best-Practices-FINAL.pdf.

———. 2025a. "2025 Annual Survey Highlights." https://api.ctia.org/wp-content/uploads/2025/09/2025-Annual-Survey-Highlights.pdf.

———. 2025b. "Messaging Security Best Practices." https://api.ctia.org/wp-content/uploads/2025/10/Messaging-Security-Best-Practices-_October-2025.pdf.

Federal Trade Commission. 2022. Federal Trade Commission Consumer Advice. July 2022. https://consumer.ftc.gov/articles/how-recognize-report-spam-text-messages.

Gottfried, Jeffrey, Eugenie Park, and Monica Anderson. 2025. "Online Scams and Attacks in America Today." *Pew Research Center*. https://www.pewresearch.org/internet/2025/07/31/online-scams-and-attacks-in-america-today/.

"How to Check If All Elements of a List Are Equal in R: Beyond Pairwise Identical() – Native Solutions & Custom Functions." 2025. TutorialPedia.org. November 21, 2025. https://www.tutorialpedia.org/blog/check-whether-all-elements-of-a-list-are-in-equal-in-r/.

Irizarry, Rafael. n.d.a. *Introduction to Data Science*. Web. 1st ed. https://rafalab.dfci.harvard.edu/dsbook/index.html.

———. n.d.b. *Introduction to Data Science: Statistics and Prediction Algorithms Through Case Studies*. Web. 2nd ed. https://rafalab.dfci.harvard.edu/dsbook-part-2/.

"K-Nearest Neighbor(KNN) Algorithm." 2026. Geeks for Geeks. March 12, 2026. https://www.geeksforgeeks.org/machine-learning/k-nearest-neighbours/.

Silge, Julia, and David Robinson. 2025. "The Tidy Text Format." Web. In *Text Mining with R: A Tidy Approach*. https://www.tidytextmining.com/tidytext.html.

"What Is F-Beta Score?" 2025. Geeks for Geeks. July 23, 2025. https://www.geeksforgeeks.org/machine-learning/what-is-f-beta-score/.

Zantvoort, Kirsten, Barbara Nacke, Dennis Görlich, Silvan Hornstein, Corinna Jacobi, and Burkhardt Funk. n.d. "Estimation of Minimal Data Sets Sizes for Machine Learning Predictions in Digital Mental Health Interventions." *Npj Digital Medicine*. https://doi.org/https://doi.org/10.1038/s41746-024-01360-w.